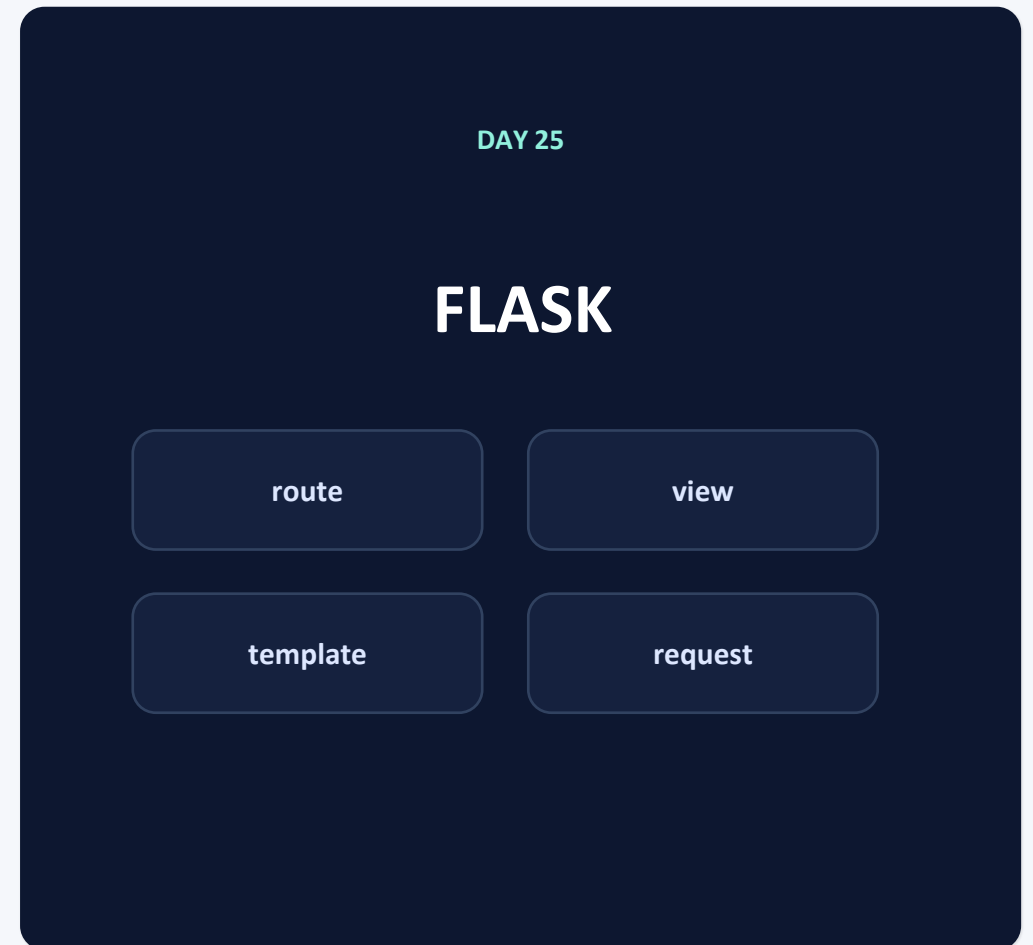


Первое веб-приложение на Flask

Собрать первое веб-приложение на Flask: приложение, route, view function, параметры URL и простой JSON/текстовый ответ.



ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Собрать первое веб-приложение на Flask: приложение, route, view function, параметры URL и простой JSON/текстовый ответ.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Frontend, backend и полный путь данных

Что уже есть в нашей модели.



СЕГОДНЯ

Первое веб-приложение на Flask

Вчера: Frontend, backend и полный путь данных. Сегодня мы добавляем следующий слой: первое веб-приложение на flask.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

**Здесь все предыдущие темы сходятся вместе:
Python-функции становятся обработчиками HTTP-
запросов.**

Вопрос себе

**Какую реальную проблему
решает сегодняшняя тема?**

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Flask-route — дверь с табличкой адреса.
Когда приходит запрос к этому адресу,
Flask вызывает привязанную функцию и
отправляет её результат клиенту.

ПЕРЕВОД В IT

Flask

лёгкий Python-фреймворк для веб-приложений

route

правило связи URL с обработчиком

view

функция, обрабатывающая веб-запрос

Слой 1: Flask • route • view

Сначала смысл, затем профессиональный термин.

**Flask**

лёгкий Python-фреймворк для веб-приложений

**route**

правило связи URL с обработчиком

**view**

функция, обрабатывающая веб-запрос

Слой 2: template • request • session

Сначала смысл, затем профессиональный термин.

**template**

HTML-заготовка с безопасными местами для данных

**request**

объект входных данных текущего HTTP-запроса

**session**

состояние пользователя между несколькими запросами

Слой 3: blueprint • context

Сначала смысл, затем профессиональный термин.

blueprint

способ группировки маршрутов Flask

context

данные, передаваемые шаблону для отрисовки

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: cookie • middleware

Сначала смысл, затем профессиональный термин.

cookie

небольшие данные, сохраняемые браузером

middleware

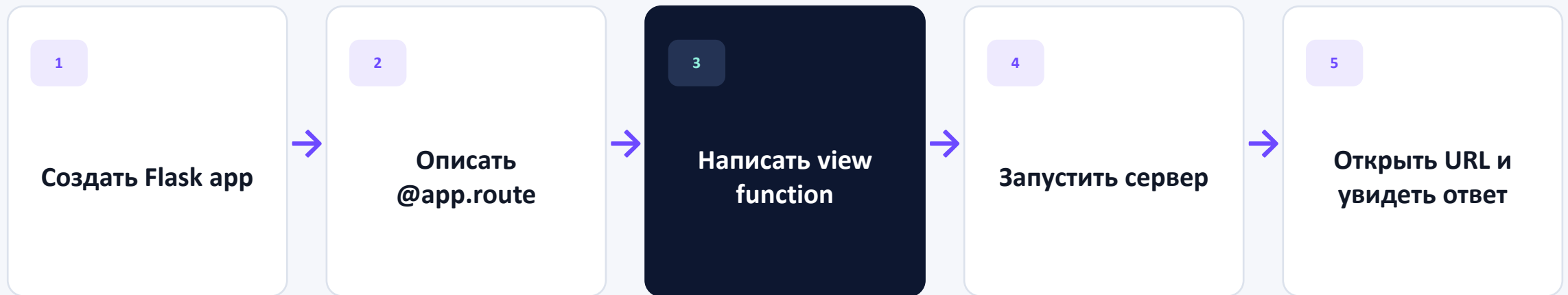
слой обработки между запросом и приложением

Не заучивай формулировку. Придумай свой пример использования каждого термина.

СХЕМА

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day25_step1

```
from flask import Flask

app = Flask(__name__)

@app.route("/hello/<name>")
def hello(name):
    return f"Привет, {name}!"
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
from flask import Flask
```

Python выполняет эту строку сверху вниз.

```
app = Flask(__name__)
```

Сохраняем значение или результат вычисления под понятным именем.

```
@app.route("/hello/<name>")
```

Python выполняет эту строку сверху вниз.

```
def hello(name):
```

Создаём функцию и даём ей имя; код внутри выполнится при вызове.

```
return f"Привет, {name}!"
```

Python выполняет эту строку сверху вниз.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
from flask import Flask

app = Flask(__name__)

@app.route("/hello/<name>")
def hello(name):
    return f"Привет, {name}!"
```

РЕЗУЛЬТАТ

GET /hello/Anna -> Привет, Anna!

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.

```
@app.route("/courses/<int:course_id>")
def course(course_id):
    return {"id": course_id, "title": "Python"}
```

changed



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

GET /courses/7 -> JSON с id=7 и title=Python

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.

```
def hello():  
    return "Hello"  
# функция есть, но route отсутствует
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

Обычная функция сама по себе не является HTTP endpoint. Flask должен знать, к какому URL её привязать.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.

```
@app.route("/hello")
def hello():
    return "Hello"
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Сделать endpoint `/courses/<int:course_id>`, который возвращает JSON курса.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Создаём app.**
- 2 Декоратором задаём URL с int-параметром.
- 3 Функция получает `course_id`.
- 4 Формируем словарь.
- 5 Flask сериализует словарь в JSON-ответ.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
from flask import Flask
app = Flask(__name__)

@app.route("/courses/<int:course_id>")
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
from flask import Flask
app = Flask(__name__)

@app.route("/courses/<int:course_id>")
def course(course_id):
    return {"id": course_id, "title": "Python", "days": 30}
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

```
GET /courses/1 -> {"id":1,"title":"Python","days":30}
```

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Создай route `/square/<int:number>`, который возвращает квадрат числа.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Параметр `number` уже `int`; вычисли `number * number` и верни строку или словарь.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
@app.route("/square/<int:number>")  
def square(number):  
    return {"number": number, "square": number * number}
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

```
GET /square/5 -> {"number":5,"square":25}
```

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Сделай два route: / и /about, чтобы увидеть, как URL выбирает функцию.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.

A dark-themed terminal window with a title bar at the top containing three colored circles (red, yellow, green) on the left and the text 'mini_project' on the right. The terminal contains two Python code blocks. The first block defines a route for the root path, and the second block defines a route for the '/about' path.

```
@app.route("/")  
def index():  
    return "Главная"  
  
@app.route("/about")  
def about():  
    return "О проекте"
```

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

Разные URL вызывают разные view functions

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01 **Забывать декоратор route.**

02 **Запускать debug=True в production.**

03 **Делать всю бизнес-логику внутри одной view function.**

04 **Доверять любым параметрам запроса без проверки.**

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «Flask» и какую проблему это понятие помогает описать?

2

Чем «Flask» отличается от «template»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **Flask — лёгкий Python-фреймворк для веб-приложений.**

2 Flask: лёгкий Python-фреймворк для веб-приложений. template: HTML-заготовка с безопасными местами для данных.

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 GET /hello/Anna -> Привет, Anna!

4 Обычная функция сама по себе не является HTTP endpoint. Flask должен знать, к какому URL её привязать.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Собрать первое веб-приложение на Flask: приложение, route, view function, параметры URL и простой JSON/текстовый ответ.
- ✓ Читать и объяснять пример темы «Первое веб-приложение на Flask».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

[ДАЛЬШЕ](#)

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Тестирование

Завтра: Тестирование. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.